

S11BLH21- Programming in python



SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

UNIT-IV S11BLH21 – Programming in python

GUI PROGRAMMING

GUI Programming in Python - Introduction to GUI library - Layout management - Events and bindings - Fonts – Colours - Canvas - Widgets (frame, label, button, check box, entry, list box, message, radio button, text, spin box)

GUI PROGRAMMING WITH PYTHON

In python text only programs can be created using Command line Interface. Graphical user interface(GUI) can be created using tkinter module in python.

Introduction To GUI Library In Python

Tkinter is a module in the Python standard library which serves as an interface to Tk (ie) simple *toolkit*. There are many other toolkits also available to create GUI.

Tkinter provides the following widgets:

- button
- canvas
- checkbutton
- combobox
- entry
- frame
- label
- listbox
- menu
- message
- progressbar
- radiobutton

S11BLH21- Programming in python

- scrollbar
- spinbox
- text

Tkinter also provides three layout managers:

- place - It positions widgets at absolute locations
- grid - It arranges widgets in a grid
- pack - It packs widgets into a cavity

Layout Management

The Layout Managers are used to arrange components in a particular manner. It is used to organize the components. There are three Layout Management in python:

1. Pack Layout
2. Grid Layout
3. Place Layout

Pack Layout Manager

It is a simple layout manager. Here widgets can be organized in horizontal and vertical boxes. It is used to place each widget next to previous widget. It will be called without any arguments and it will position and size the widgets in a reasonable way. Whenever the user wants to have a series of widgets in a vertical or horizontal row, the pack layout manager is fairly simple to use. The layout is controlled with the fill, expand, and side options.

S11BLH21- Programming in python

Example:

```
from tkinter import *
top=Tk()
l1=Label(top,text="Label1",bg="blue")
l2=Label(top,text="Label2",bg="red" )
l1.pack(fill=X,side=TOP,expand=True)
l2.pack(fill=X,side=RIGHT)
top.mainloop()
```

Output:

The output is shown in Fig 3.1.



Fig 3.1

Explanation: Label l1 has been placed in top position, it is filled in X axis. Label l2 has been placed in Right Position and it is also filled in X axis. Since expand attribute has the value True for Label l1,it can be stretched.

Padding Option in Pack Layout:

The pack() manager has four padding options:

1. Internal Padding
2. External padding
3. Padding in X Direction.
4. Padding in Y Direction.

External Padding in Horizontal direction (padx)

Example:

```
from tkinter import *  
top=Tk()  
l1=Label(top,text="Label1",bg="blue")  
l2=Label(top,text="Label2",bg="red" )  
l1.pack(fill=X,side=TOP,expand=True,padx=10)  
l2.pack(fill=X,side=TOP,padx=10)  
top.mainloop()
```

Output:



Fig 3.2

External Padding in Vertical direction (pady)

Example:

```
from tkinter import *
top=Tk()

l1=Label(top,text="Label1",bg="blue")
l2=Label(top,text="Label2",bg="red" )

l1.pack(fill=X,side=TOP,expand=True,pady=10)
l2.pack(fill=X,side=TOP,pady=10)

top.mainloop()
```

Output:

The output is shown in Fig 3.3.



Fig 3.3

Internal Padding in Horizontal direction(ipadx)

Example:

```
from tkinter import *  
top=Tk()  
l1=Label(top,text="Label1",bg="blue")  
l2=Label(top,text="Label2",bg="red" )  
l1.pack(fill=X,side=TOP,expand=True,ipadx=10)  
l2.pack(fill=X,side=TOP,ipadx=10)  
top.mainloop()
```

Output:

The output is shown in Fig 3.4.



Fig 3.4

Internal Padding in Y Direction(ipady):

Example:

```
from tkinter import *  
top=Tk()  
l1=Label(top,text="Label1",bg="blue")  
l2=Label(top,text="Label2",bg="red" )  
l1.pack(fill=X,side=TOP,expand=True,ipadx=10)  
l2.pack(fill=X,side=TOP,ipady=10)  
top.mainloop()
```

Output:

The output is shown in Fig 3.5.



Fig 3.5

Place Layout:

Place is the most complex manager out of the 3 managers. It uses absolute positioning, when we choose place lay out in our design, then we need to specify the position of the widgets using x and y coordinates. The size and position of the widgets will not be changed when we resize the window.

Example:

```
from tkinter import *  
  
top=Tk()  
  
l1=Label(top,text="Label1",bg="blue")  
l2=Label(top,text="Label2",bg="red" )  
  
l1.place(x=10,y=50)  
l2.place(x=10,y=100)  
  
top.mainloop()
```

Output:

The output is shown in Fig 3.6.

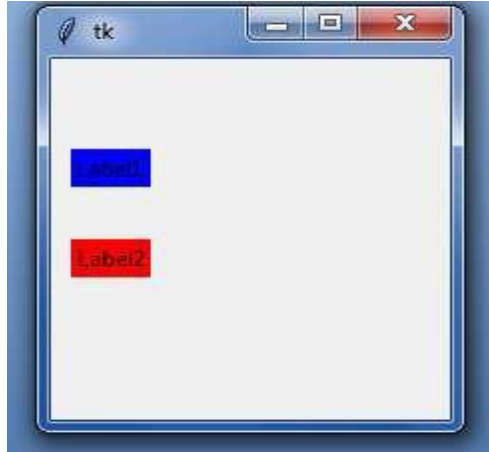


Fig 3.6

Explanation:

Here Label1 is placed in the position (10,50) and label2 is placed in the position (10,100).

Grid Layout

Pack Layout is not easy to understand and it is difficult to change the existing design. By using place layout, we can control the positioning of widgets but it is complex than pack. Grid is one of the most versatile layout manager out of the three layout managers in python. By using Grid layout, the widgets can be placed in rows and columns.

Example:

```
from tkinter import *
top=Tk()
l1=Label(top,text="Label1",bg="blue")
l2=Label(top,text="Label2",bg="red" )
l3=Label(top,text="Label2",bg="green" )
l1.grid(row=0,column=0)
l2.grid(row=0,column=1)
l3.grid(row=1,column=1)
top.mainloop()
```

Output:

The output is shown in Fig 3.7.

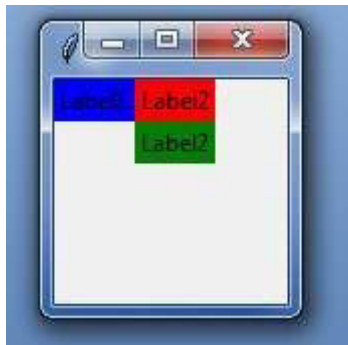


Fig 3.7

S11BLH21- Programming in python

Explanation:

Here Label 1 is placed in 0th row and 0th column. Label 2 is placed in 0th row and 1st column and Label 3 is placed in 1st row and 1st column.

FONT

There are three ways to specify font in python.

- 1.By using Font Tuple
- 2.By using Font Object
- 3.By using XFont

Simple Font Tuple:

Font can be specified using tuple. Here the font tuple consists of three elements. First element specifies font family, second element specifies font size and third element specifies font style.

Ex: t=("Arial",14,"Bold")

Example:

```
from tkinter import *
top=Tk()
b1=Button(text="submit",font=("Arial","16","bold"))
b1.pack()
top.mainloop()
```

Output:

The output is shown in Fig 3.8.



Fig 3.8

Explanation:

Text for the Button has been set in the Arial font with size 16 and Bold style.

Font Object

Font object can be created by importing tkFont module.

Syntax for Font class constructor is:

Import tkFont

Font f1=tkFont.Font(parameters,.....)

Here is the list of parameters:

S11BLH21- Programming in python

Family	– The font family name as a string.
size	– The font height as an integer in points. To get a font n pixels high, use -n.
weight	– "bold" for boldface, "normal" for regular weight.
Slant	– "italic" for italic, "roman" for unslanted.
underline	– 1 for underlined text, 0 for normal.
Overstrike	– 1 for overstruck text, 0 for normal

Example:

```
from tkinter import *
from tkFont import *
top=Tk()
f1=Font(family="Helvetica",size=20,weight="bold",slant="italic",underline=1,overstrike=1)
l1=Label(top,text="Label1",bg="blue",font=f1)
l1.pack()
top.mainloop()
```

X Window Fonts:

If you are running under the X Window System, you can use any of the X font names.

COLORS

Tkinter represents colors with strings. There are two general ways to specify colors in Tkinter :

- We can use a string specifying the proportion of red, green and blue in hexadecimal digits. For example,
 - "#fff" -- white,
 - "#000000" -- black,
 - "#000fff000" -- pure green
 - "#00ffff" -- pure cyan
- We can also use any locally defined standard following color names.
 - "white"
 - "black"
 - "red"
 - "green"
 - "blue"
 - "cyan"
 - "yellow"
 - "magenta"

S11BLH21- Programming in python

The common color options are :

- | | |
|---------------------|--|
| Active background | – Specifies Background color for the widget when the widget is active. |
| activeforeground | – Specifies Foreground color for the widget when the widget is active. |
| background | – Specifies Background color for the widget. This can also be represented as bg. |
| disabledforeground | – Specifies Foreground color for the widget when the widget is disabled. |
| foreground | – Specifies Foreground color for the widget. This can also be represented as fg. |
| highlightbackground | – Specifies Background color of the highlight region when the widget has focus. |
| highlightcolor | – Specifies Foreground color of the highlight region when the widget has focus. |
| selectbackground | – Specifies Background color for the selected items of the widget. |
| selectforeground | – Specifies Foreground color for the selected items of the widget. |

S11BLH21- Programming in python

Example:

```
from tkinter import *  
top=Tk()  
b1=Button(text="submit",bg="red",fg="white")  
b1.pack()  
top.mainloop()
```

Output:

The output is shown in Fig 3.9.

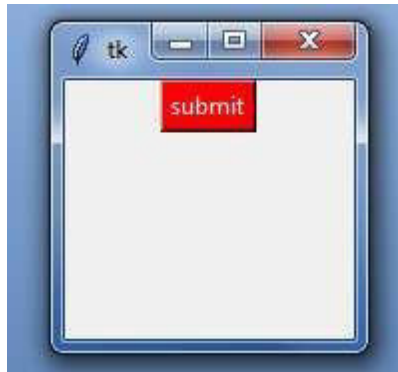


Fig 3.9

Explanation:

Here the back ground of the button is red in color and foreground color of the button is white in colour.

CANVAS

The Canvas is a rectangular area used for drawing pictures or other complex layouts. Graphics, text, widgets or frames can be placed on a Canvas.

Syntax:

w = Canvas (top, option=value, ...)

top – It represents the parent window.

Options – commonly used options for this widget. These options can be used as key-value pairs separated by commas.

Commonly used Options are:

bd - Border Width of the canvas

bg - Background color of the canvas

cursor - Cursor used in the canvas like circle, arrow and dot.

relief - Type of the border

width - Width of the canvas

Items supported by canvas:

1.Arc

2.Image

3.Line

4.Oval

5.Polygon

S11BLH21- Programming in python

ARC

Creates an arc item, which can be a chord or a simple arc.

Syntax:

`create_arc(x0, y0, x1, y1, options.....)`

`x0,y0,x1,y1`-Top Left and Bottom Right coordinates of Bounding Rectangle

Commonly used Options:

`start,extent`-Specifies which section to draw

Example:

```
from tkinter import *
root=Tk()
w = Canvas(root, width=500, height=500)
coord = 10, 50, 240, 210
arc = w.create_arc(coord, start=0, extent=150, fill="blue")
w.pack()
```

Output:

The output is shown in Fig 3.10.

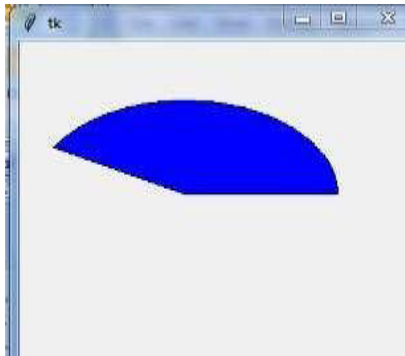


Fig 3.10

Explanation:

Here Arc is drawn with blue color and within the bounded rectangle with top left(10,50)position and bottom right(240,210) position and started from angle 0 and extended till 150 degree.

Image

Creates an image , which can be an instance of either the BitmapImage or the PhotoImage classes.

Syntax:

Create_image(x,y,options....)

x,y-Specifies the position of the image

commonly used options:

S11BLH21- Programming in python

anchor=Where to place the image relative to the given position. Default is CENTER.

image=image object

Example:

```
from tkinter import *  
root=Tk()  
w = Canvas(root, width=500, height=500)  
w.create_image("F:\img2",50,50)  
w.pack()  
root.mainloop()
```

Line

Creates a line item.

Syntax:

canvas.create_line(x0, y0, x1, y1, ...,xn, yn, options)

x0,y0,x1,y1->coordinates of line

Commonly used options:

activefill-Color of the line when it is active

width -Width of the line

Example:

```
from tkinter import *  
root=Tk()  
w = Canvas(root, width=500, height=500)  
w.create_line(10,10,100,100,activefill="red")  
w.pack()  
root.mainloop()
```

Output:

The output is shown in Fig 3.11.

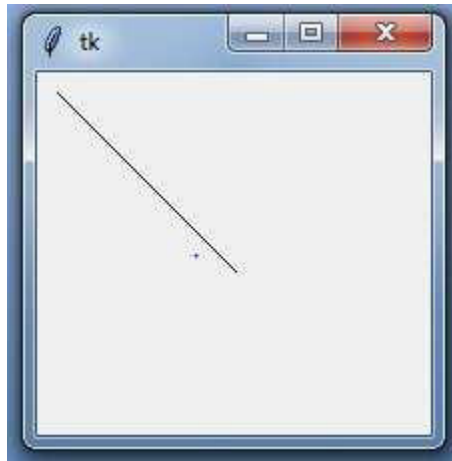


Fig 3.11

OVAL

Creates a circle or an ellipse at the given coordinates. It takes two pairs of coordinates; the top left and bottom right corners of the bounding rectangle for the oval.

Syntax:

```
canvas.create_oval(x0, y0, x1, y1, options)
```

x0, y0, x1, y1- the top left and bottom right corners of the bounding rectangle

Options:

activefill-Color of the oval when it is active

width -Width of the line

Example:

```
from tkinter import *
root=Tk()
w = Canvas(root, width=500, height=500)
w.create_oval(10,10,100,100,activefill="red")
w.pack()
root.mainloop()
```

Output:

The output is shown in Fig 3.12.

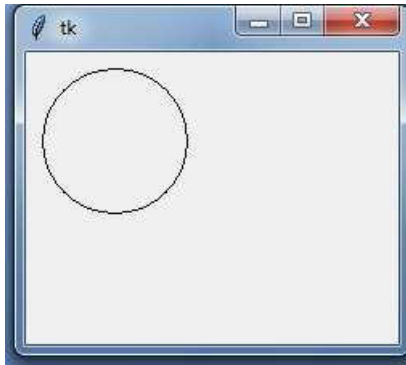


Fig 3.12

Polygon

Creates a polygon item that must have at least three vertices.

Syntax:

```
canvas.create_polygon(x0, y0, x1, y1,...xn, yn, options)
```

x0, y0, x1, y1,...xn, yn-Coordinates of polygon

Options:

Activefill-Color of the oval when it is active

width -Width of the line

Example

```
from tkinter import *
root=Tk()
w = Canvas(root, width=500, height=500)
w.create_polygon(50,50,20,20,100,100,activefill="red")
w.pack()
root.mainloop()
```

WIDGETS IN PYTHON

Widgets are standard graphical user interface (GUI) elements, like different kinds of buttons and menus.

Label

A Label widget shows text to the user about other widgets used in the application. The widget can be updated programmatically.

Syntax to create Label:

```
w=Label (root ,options)
```

root - Parent Window

List of commonly used options are given in Table 3.1.

Table 3.1

Option	Description
anchor	It specifies the exact position of the text within the size provided to the widget. The default value is CENTER, which is used to center the text within the specified space.
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
text	Text to be displayed in the widget
underline	Underline the label text

Example:

```
from tkinter import *  
root=Tk()  
l1=Label(root,text="Enter User Name",bg="green",fg="white")  
l1.pack()  
root.mainloop()
```

Output:

The output is shown in Fig 3.13.

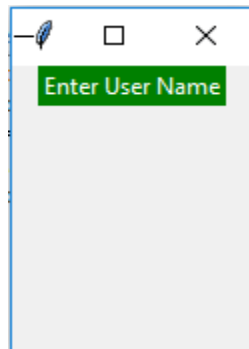


Fig 3.13

Explanation:

Here Label has been created with green background color and white foreground color with the text “Enter User Name”.

ENTRY

The Entry widget is used to create the single line text-box to the user to accept a value from the user. It can accept the text strings from the user. It can receive one line of text from the user. For multiple lines of text, the text widget will be used.

Syntax for creating Entry Widget:

```
w=Entry(root, options)
```

root-Main Window

List of commonly used options are given in Table 3.2

Table 3.2

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor. eg: dot, arrow, circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget

S11BLH21- Programming in python

width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text
pady	Vertical padding of text

S11BLH21- Programming in python

Option	Description
relief	Specifies type of border
text	Text to be displayed in the widget
underline	Underline the label text
selectbackground	Background color of the selected text
selectforeground	Foreground color of the selected text
show	Specifies the character used to mask characters in the text box

Example:

```
from tkinter import *
root=Tk()
l1=Label(root,text="Enter User Name",bg="green",fg="white")
e1=Entry(root,show="*")
l1.pack(side=LEFT)
e1.pack(side=RIGHT)
root.mainloop()
```

Output:

The output is shown in Fig 3.14.

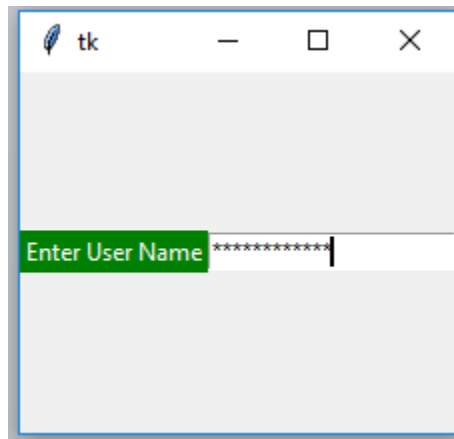


Fig 3.14

Explanation:

Here Label and entry widgets are created. Since the show attribute value is *, the characters entered in the text box appeared as “*”.

Button

Button Widget is used to create various kinds of buttons. The user can interact with the button. They can contain text or images.

Syntax for creating Button:

```
b=Button(root,options)
```

root-main window

List of commonly used options are given in Table 3.3

Table 3.3

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
text	Text to be displayed in the widget
underline	Underline the label text
command	It is set to function name which will be called the button is clicked

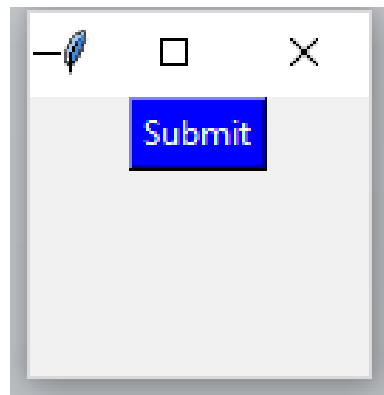
Example:

S11BLH21- Programming in python

```
from tkinter import *  
root=Tk()  
b1=Button(root,text="Submit",bg="blue",fg="white")  
b1.pack()  
root.mainloop()
```

Output:

The output is shown in Fig 3.15.



Checkbutton

The Checkbutton is used to track the user's choices provided to the application. Checkbutton is used to implement the on/off selections. TheCheckbutton can contain the or images or text. The Checkbutton is mostly used to provide many choices to the user among which, the user needs to choose the one.

Syntax for creating Check Button:

```
b=CheckButton(root,options)
```

root-main window

S11BLH21- Programming in python

List of commonly used options are given in Table 3.2

Table 3.2

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
text	Text to be displayed in the widget
underline	Underline the label text
command	It is set to function name which will be called the button is clicked
offvalue	Set value to the control variable if the button is checked.Default Value is 1
onvalue	Set value to the control variable if the button is unchecked.Default Value is 0
selectcolor	Set color of the check button when it is checked.
selectimage	Set the image to be shown when it is checked.

Example:

```
from tkinter import *
root=Tk()
c1 = Checkbutton(root, text = "C", onvalue = 1, offvalue = 0, height = 2, width = 10)
c2 = Checkbutton(root, text = "C++", onvalue = 1, offvalue = 0, height = 2, width = 10)
c3 = Checkbutton(root, text = "JAVA", onvalue = 1, offvalue = 0, height = 2, width = 10)
c1.pack()
c2.pack()
c3.pack()
root.mainloop()
```

Output:

The output is shown in Fig 3.16.



Fig 3.16

Radiobutton

The Radiobutton widget is used to implement one-of-many selection. It shows multiple options to the user out of which, the user can select only one option. It is possible to display the multiple line text or images on the radiobuttons. To keep track the user's selection ,theradiobutton is associated with a single variable.EachRadio button displays a single value for that particular variable.

Syntax for creating Radio Button:

```
b=RadioButton(root,options)
```

root-main window

List of commonly used options are given in Table 3.3

Table 3.3

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget

S11BLH21- Programming in python

Option	Description
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
text	Text to be displayed in the widget
underline	Underline the label text
command	It is set to function name whicjh will be called the button is clicked
value	Set value to the control variable if the button is selected.
selectcolor	Set color of the check button when it is checked.
selectimage	Set the image to be shown when it is checked.
variable	It is used to keep track of user choices.

Example:

```
from tkinter import *
root=Tk()
r1 = Radiobutton(root, text = "C", value = 1, height = 2, width = 10)
r2 = Radiobutton(root, text = "C++", value = 2, height = 2, width = 10)
r3 = Radiobutton(root, text = "JAVA",value = 3, height = 2, width = 10)
r1.pack()
r2.pack()
r3.pack()
root.mainloop()
```

Output:

The output is shown in Fig 3.17.

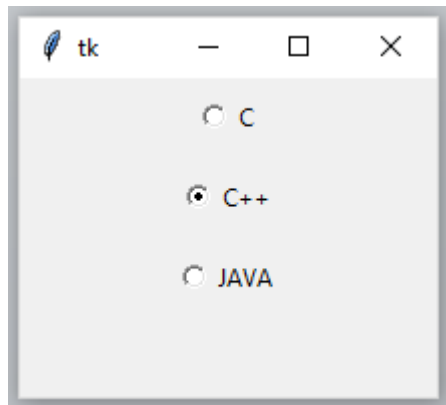


Fig 3.17

Listbox

The Listbox widget is used to display the list items to the user. The user can choose one or more items from the list depending upon the configuration.

Syntax for creatingListBox:

```
b=Listbox(root,options)
```

root-main window

Table 3.4

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
value	Set value to the control variable if the button is selected.
selectbackground	Set back ground color of the selected text.
xscrollcommand	User can scroll the list box horizontally
yscrollcommand	User can scroll the list box vertically

List of commonly used options are given in Table 3.4.

Example:

```
from tkinter import *
top = Tk()
lbl = Label(top, text = "A list of favourite countries...")
listbox = Listbox(top)
listbox.insert(1, "India")
listbox.insert(2, "USA")
listbox.insert(3, "Japan")
listbox.insert(4, "Austrelia")
lbl.pack()
listbox.pack()
top.mainloop()
```

Output:

The output is shown in Fig 3.18.



Message

Its functionality is very similar to Label widget, except that it can automatically wrap the text, maintaining a given width.

Syntax for creating Message:

```
m=Message(root,options)
```

root-main window

List of commonly used options are given in Table 3.5.

Table 3.5

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text

Option	Description
pady	Vertical padding of text
relief	Specifies type of border

Example:

```
from tkinter import *  
top = Tk()  
msg = Message( top, text = "Welcome to Javatpoint")  
msg.pack()  
top.mainloop()
```

Output:

The output is shown in Fig 3.19.

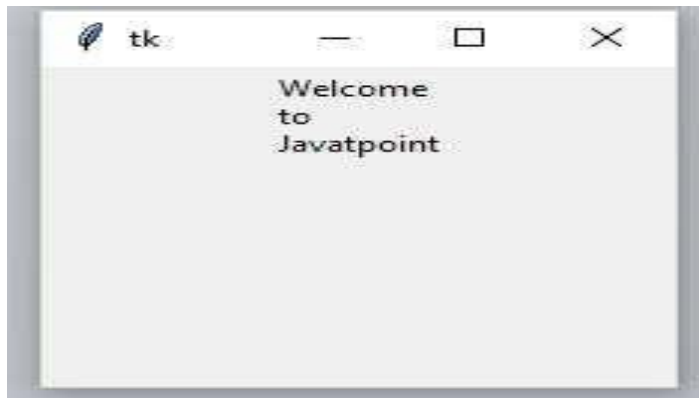


Fig 3.19

Text

Tkinter provides us the Entry widget which is used to implement the single line text box. Text widget provides advanced capabilities that allow us to edit a multiline text and format the way it has to be displayed, such as changing its color and font. We can also use the structures like tabs and marks to locate specific sections of the text, and apply changes to those areas.

Syntax for creating Message:

```
T=Text(root,options)
```

root-main window

List of commonly used options are given in Table 3.6.

Table 3.6

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget
height	Height of the widget
width	Width of the widget
image	Specifies image to be displayed in the widget
padx	Horizontal padding of text

S11BLH21- Programming in python

Option	Description
pady	Vertical padding of text
relief	Specifies type of border
xscrollcommand	User can scroll the text widget horizontally
yscrollcommand	User can scroll the text widget vertically
selectbackground	Background color of the selected text

General Methods are given in Table 3.7.

Table 3.7

Method	Description
delete(startindex, endindex)	This method is used to delete the characters of the specified range
get(startindex,endindex)	It returns the characters present in the specified range.
insert(index, string)	It is used to insert the specified string at the given index.

Mark Handling Methods :

Marks are used to bookmark the specified position between the characters of the associated text. List of Mark handling methods are given in Table 3.8.

Table 3.8

Method	Description
mark_set(mark,index)	It is used to create mark at the specified index.
mark_unset(mark)	It is used to clear the given mark
mark_names()	It is used to return names of all the marks

Tag Handling Methods:

The tags are the names given to the specific areas of the text. The tags are used to configure the different areas of the text separately. The list of tag-handling methods are given in Table 3.9.

Table 3.9

Method	Description
tag_add(tagname, startindex, endindex)	It is used to tag the characters in the given range
tag_config()	It is used to configure the tag properties
tag_delete(tagname)	It is used to delete the given tag
tag_remove(tagname, startindex, endindex)	It is used to remove the tag from the specified range

Example:

```
from tkinter import *
top = Tk()
text = Text(top)
text.insert(INSERT, "Name ....")
text.insert(END, "Salary ....")
text.pack()
text.tag_add("Write Here", "1.0", "1.4")
text.tag_add("Click Here", "1.8", "1.13")
text.tag_config("Write Here", background="yellow", foreground="black")
text.tag_config("Click Here", background="black", foreground="white")
```

Output:

The output is shown in Fig 3.20.

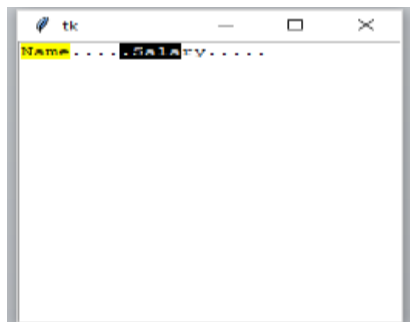


Fig 3.20

Explanation:

The tag “Write Here” tags the characters from the index 0 to 4. The tag “Click Here” tags the characters from the index 8 to 13. These tags are configured using the method `tag_config()`.

3.6.1 SPINBOX

The Spinbox control is an alternative to the Entry control. It provides the range of values to the user, out of which, the user can select only one value. It is used in the case where a user is given some fixed number of values to choose from.

Syntax for creating Message:

`S=Spinbox(root,options)`

root-main window

List of commonly used options are given in Table 3.10

Table 3.10

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor. eg: dot, arrow, circle
font	Specifies font type of the text written inside the widget
fg	Foreground color of the widget

S11BLH21- Programming in python

Option	Description
height	Height of the widget
width	Width of the widget
padx	Horizontal padding of text
pady	Vertical padding of text
relief	Specifies type of border
xscrollcommand	User can scroll the text widget horizontally
from_	It is used to show the starting range of the widget.
to	It specify the maximum limit of the widget value. The other is specified by the from_ option.
values	It represents the tuple containing the values for this widget.

Example:

```
from tkinter import *
top = Tk()
spin = Spinbox(top, from_ = 0, to = 25)
spin.pack()
top.mainloop()
```

Output:

The output is shown in Fig 3.21.



Fig 3.21

3.6.2 Frame

Frame widget is used to organize the group of widgets. It acts like a container which can be used to hold the other widgets. The rectangular areas of the screen are used to organize the widgets to the python application.

Syntax for creating Frame:

```
S=Frame(root,options)
```

root-main window

List of commonly used options are given in Table 3.11.

Table 3.11

Option	Description
bg	Specifies background color of the widget
bd	Specifies border width. Default is 2 pixels
cursor	Specifies type of cursor.eg:dot,arrow,circle
height	Height of the widget
width	Width of the widget
Relief	Specifies type of border

Example:

```
from tkinter import *
top = Tk()
Topframe = Frame(top)
Topframe.pack(side = TOP)
Bottomframe = Frame(top)
Bottomframe.pack(side =BOTTOM)
btn1 = Button(Topframe, text="Submit", fg="red",activebackground = "red")
btn1.pack(side = LEFT)
btn2 = Button(Topframe, text="Remove", fg="brown", activebackground = "brown")
btn2.pack(side = RIGHT)
btn3 = Button(Bottomframe, text="Add", fg="blue", activebackground = "blue")
btn3.pack(side = LEFT)
```

S11BLH21- Programming in python

```
btn4 = Button(Bottomframe, text="Modify", fg="black", activebackground =  
"white")  
btn4.pack(side = RIGHT)  
top.mainloop()
```

Output:

The output is shown in Fig 3.23.



Fig 3.23

Explanation:

Here two frames (Top Frame and Bottom Frame) have been created. Topframe contains submit and remove buttons and Bottom frame contains Add and modify buttons .

EVENTS AND BINDINGS IN PYTHON

Binding function is used to deal with the events. We can bind Python's Functions and methods to an event as well as we can bind these functions to any particular widget. Events can come from various sources, including key presses and mouse operations by the user. Tkinter provides a powerful mechanism to let you deal with events yourself. For each widget, you can bind Python functions and methods to events.

```
widget.bind(event, handler)
```

If an event matching the event description occurs in the widget, the given handler is called with an object describing the event.

Handling Mouse Button event in Python

Example:

```
from tkinter import *
from tkinter.ttk import *
# creates tkinter window or root window
root = Tk()
# function to be called when button-2 of mouse is pressed
def pressed2(event):
    print('Button-2 pressed at x = % d, y = % d'%(event.x, event.y))
# function to be called when button-3 of mouse is pressed
def pressed3(event):
    print('Button-3 pressed at x = % d, y = % d'%(event.x, event.y))
## function to be called when button-1 is double clicked
defdouble_click(event):
```

S11BLH21- Programming in python

```
print('Double clicked at x = % d, y = % d'%(event.x, event.y))
frame1 = Frame(root, height = 100, width = 200)
# Binding mouse buttons with the Frame widget
frame1.bind('<Button-2>', pressed2)
frame1.bind('<Button-3>', pressed3)
frame1.bind('<Double 1>', double_click)
frame1.pack()
root.mainloop()
```

Output:

The output is shown in Fig 3.24.

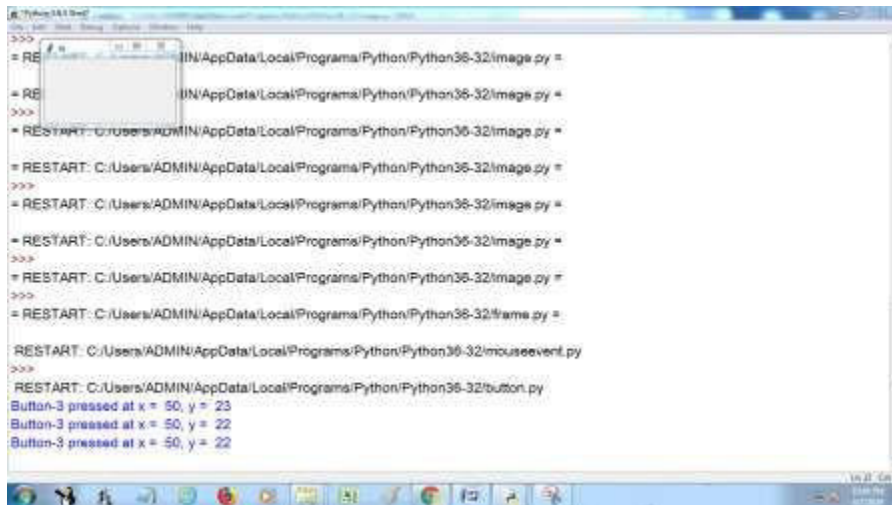


Fig 3.24

Handling Key Press Event in Python

Example:

```
from tkinter import *
from tkinter.ttk import *

# function to be called when
# keyboard buttons are pressed
def key_press(event):
    key = event.char
    print(key, 'is pressed')

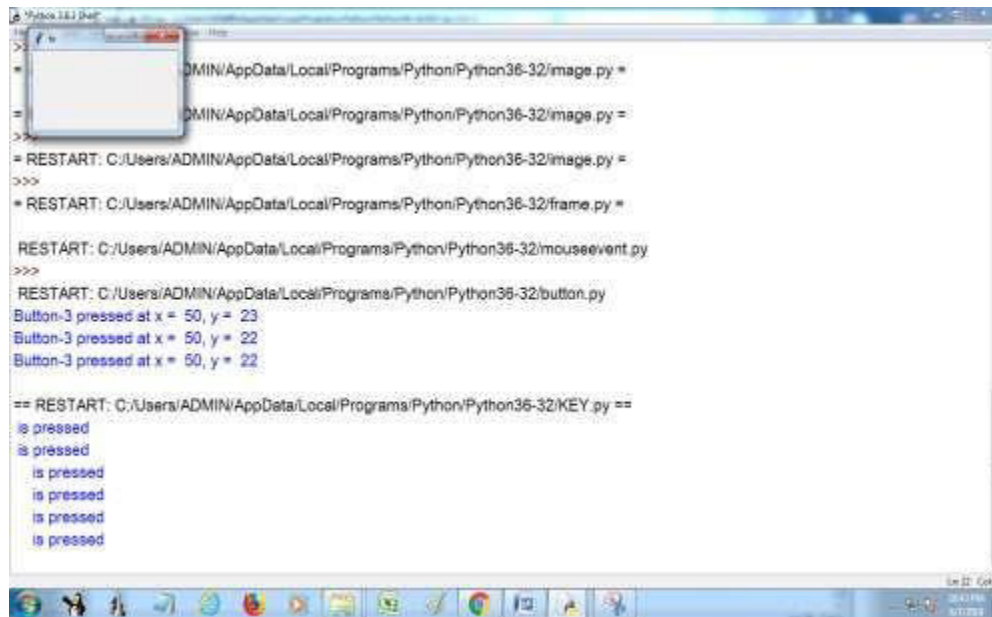
# creates tkinter window or root window
root = Tk()
root.geometry('200x100')

# here we are binding keyboard
# with the main window
root.bind('<Key>', lambda a : key_press(a))

mainloop()
```

Output:

The output is shown in Fig 3.25.



```
>>>
= RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/image.py =
>>>
= RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/image.py =
>>>
= RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/frame.py =
>>>
= RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/mouseevent.py
>>>
= RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/button.py
Button-3 pressed at x = 50, y = 23
Button-3 pressed at x = 50, y = 22
Button-3 pressed at x = 50, y = 22

== RESTART: C:/Users/ADMIN/AppData/Local/Programs/Python/Python36-32/KEY.py ==
is pressed
is pressed
  is pressed
  is pressed
  is pressed
  is pressed
```

Fig 3.25